Apuntes · Clase 47 · Embeddings y representaciones vectoriales

Para leer antes de los ejercicios · pensado para quien sabe Python pero nunca ha visto un embedding

Lo más importante, en 8 ideas

1. **Texto → vector.** un computador compara números: hay que convertir cada texto en un vector del mismo largo.
2. **Conteo.** cuenta palabras; sirve para coincidencias exactas; los sinónimos quedan perpendiculares.
3. **TF-IDF.** conteo × peso de rareza, IDF = ln((1+N)/(1+DF)) + 1.
4. **Embedding.** vector aprendido (MiniLM: 384 números) que deja cerca los textos de significado parecido.
5. **Coseno.** compara direcciones: $q^T z/(\|q\|\|z\|)$; con vectores normalizados es solo la suma de productos.
6. **Ranking.** se ordenan las 12 fichas por puntaje; el rango r es la posición de la correcta.
7. **Hit@k y MRR.** Hit@k: ¿está entre las k primeras?; MRR: promedio de 1/r.
8. **Límites.** códigos casi iguales, información insuficiente y consultas sin respuesta: recuperar no es responder.

Cada sección trae una explicación en palabras simples, un **ejemplo resuelto con números distintos** a los de las diapositivas y los ejercicios, y un resumen «para recordar». Ejercicios en `C47_Ejercicios_Cuaderno_Embeddings.pdf` y notebook `C47_Embeddings.ipynb`.

1 Convertir un texto en un vector: contar palabras

Un computador solo calcula con números. Para comparar dos textos, lo primero es **darle a cada texto una lista de números del mismo largo**. La receta más simple: elegir un **vocabulario** (por ejemplo, las palabras que aparecen en las fichas) y, para cada texto, contar cuántas veces sale cada palabra. Cada texto queda como una fila de una tabla; cada columna es una palabra. El orden de las palabras se pierde (por eso se llama «bolsa de palabras»). Y como todas las filas comparten las columnas, se pueden multiplicar, restar y comparar.

Ejemplo resuelto · tres textos y cuatro palabras (ejemplo propio)

Texto	clave	cuota	pagar	recuperar
«pagar cuota»	0	1	1	0
«pagar clave»	1	0	1	0
«recuperar clave clave»	2	0	0	1
Producto interno de los textos 1 y 2				1
Producto interno de los textos 2 y 3				2
Producto interno de los textos 1 y 3				0

Lectura: los textos 1 y 2 comparten «pagar»; los textos 2 y 3, «clave»; los textos 1 y 3 no comparten ninguna palabra y su producto es 0. El coseno confirma lo mismo sin el efecto del largo: textos 1 y 2: 0.5000; textos 2 y 3: 0.6325; textos 1 y 3: 0.0000. **Error común**: creer que el vector «entiende» el texto; solo cuenta coincidencias de palabras exactas. Si el texto 3 dijera «contraseña» en lugar de «clave», no se parecería al texto 2.

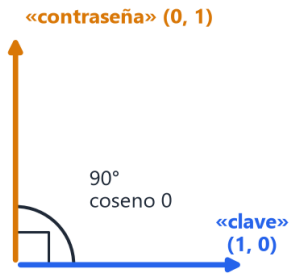
Contar palabras: un vector por texto

texto

	clave	contraseña	video
«clave»	1	0	0
«contraseña»	0	1	0
«video clave»	1	0	1

Cada columna es una palabra del vocabulario.

Sinónimos = flechas perpendiculares



Lo que debes recordar

- Vocabulario fijo → todos los textos tienen el mismo largo
- Conteo = coincidencias exactas; sin orden y sin significado
- Sinónimos distintos = coordenadas distintas = coseno 0

2 TF-IDF: darle más peso a lo raro, paso a paso

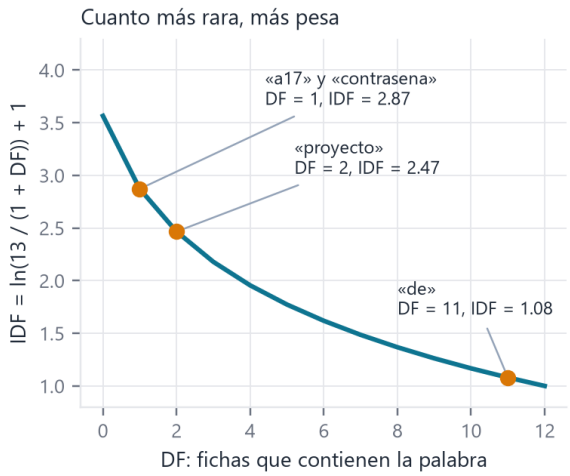
Contar da el mismo valor a «de» que a «A17», aunque «de» aparece en casi todas las fichas y «A17» solo en una. **TF-IDF** corrige eso en tres pasos: (1) contar (TF); (2) calcular para cada palabra su **IDF = ln((1 + N)/(1 + DF)) + 1**, donde N es la cantidad de fichas y DF en cuántas aparece la palabra; (3) multiplicar y **normalizar** (dividir por la norma) para que cada vector mida 1. Después se compara con el coseno. Las palabras de la consulta que no están en el vocabulario se ignoran.

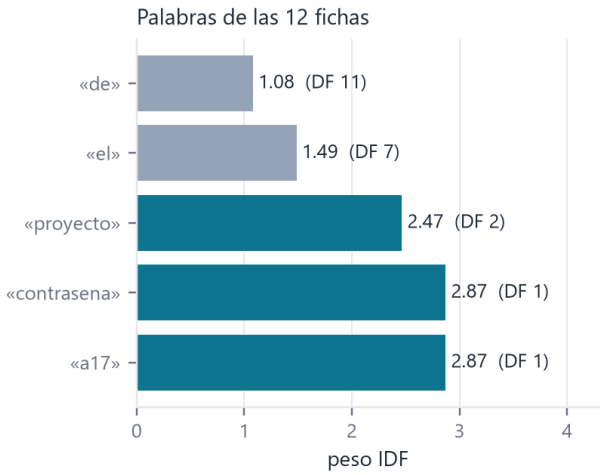
Ejemplo resuelto · 3 fichas y una consulta (ejemplo propio): «recuperar clave»

Paso	Cálculo	Resultado
DF de cada palabra	clave está en 2 fichas; cuota en 1; pagar en 2; recuperar en 1	DF = 2, 1, 2, 1
IDF (N = 3)	ln(4/(1 + DF)) + 1: ln(4/3) + 1; ln(4/2) + 1	clave 1.2877; cuota 1.6931; pagar 1.2877; recuperar 1.6931
Ficha 1 «recuperar clave»	1·1.2877 y 1·1.6931; norma $\sqrt{(1.2877^2 + 1.6931^2)}$	norma 2.1272; normalizado (0.6053; 0; 0; 0.7960)
Ficha 3 «pagar clave»	1·1.2877 (clave) y 1·1.2877 (pagar); norma 1.2877·√2	norma 1.8211; normalizado (0.7071; 0; 0.7071; 0)
Coseno de la consulta con cada ficha	producto de los vectores normalizados	ficha 1: 1.0000; ficha 2: 0.0000; ficha 3: 0.4280

Lectura: la ficha 1 es idéntica a la consulta (coseno 1); la ficha 3 comparte solo «clave» (0.4280); la ficha 2 no comparte nada (0). Fíjate que «recuperar» (IDF 1.6931) pesa más que «clave» (1.2877) porque es más rara. **Error común**: olvidar que DF cuenta fichas, no repeticiones (si «clave» aparece 3 veces en la misma ficha, DF sigue valiendo 1 por esa ficha).

En la clase (con las 12 fichas, N = 12): «a17» está en 1 ficha (IDF 2.8718), «proyecto» en 2 (2.4663), «el» en 7 (1.4855) y «de» en 11 (1.0800). La consulta V11 «¿Cuándo vence el proyecto A17?» comparte con D11 tres palabras: «a17» (0.706 × 0.244 = 0.172), «el» (0.365 × 0.252 = 0.092) y «proyecto» (0.606 × 0.209 = 0.127). La suma es 0.391: es el puntaje TF-IDF de V11 con D11 (nuestro cálculo con el programa del profesor).





Lo que debes recordar

- TF-IDF = TF × IDF, luego normalizar
- IDF alto = palabra en pocas fichas; IDF cerca de 1 = palabra en casi todas
- Solo ve palabras exactas: no entiende sinónimos

3 Embedding: un vector aprendido donde los sinónimos quedan cerca

Un **embedding** es un vector que una red neuronal ya entrenada entrega para un texto. La red se entrenó antes (con muchos textos externos y parejas marcadas como «paráfrasis» o «relevante») para que textos con el mismo sentido terminen en puntos cercanos. En la clase se usa **MiniLM**, que entrega **384 números** por texto. Hoy solo se hace **inferencia**: los pesos de la red no cambian, y los vectores de las fichas y de las consultas ya vienen calculados en `vectores.csv`. Para entender la idea sin 384 dimensiones, mira un caso de 2 números con valores inventados.

Ejemplo resuelto · embedding ilustrativo de 2 coordenadas (valores inventados, no son de MiniLM)

Texto	Vector (e_1, e_2)	Coseno con «clave»
«clave»	(0.9; 0.3)	1.0000
«contraseña»	(0.8; 0.4)	0.9899
«video»	(0.1; 0.95)	0.4138

Lectura: con conteos, «clave» y «contraseña» tenían coseno 0; con un embedding bien entrenado quedan cerca (coseno alto) y «video», que habla de otra cosa, queda lejos. Las dos coordenadas no significan «clave» ni «video»: son números que la red aprendió. **Error común**: pensar que MiniLM se reentrena con las fichas de Aula Abierta; no se reentrena, se usa tal cual. Y no se pueden comparar vectores de dos codificadores distintos, aunque tengan el mismo largo.

Destilación. Para no entrenar desde cero, una red «estudiante» aprende a imitar los vectores de un «maestro» ya entrenado. Si el maestro entrega (0.2, 0.8) y el estudiante parte en (0.4, 0.6), el error medio cuadrático es $((0.4 - 0.2)^2 + (0.6 - 0.8)^2)/2 = 0.04$; la retropropagación ajusta los pesos del estudiante para reducirlo (diapositiva 11). MiniLM es una red pequeña que imitó a una mayor.

Lo que debes recordar

- Embedding: z = f_θ(x), 384 números por texto en MiniLM
- El aprendizaje fue previo; hoy los pesos no cambian
- Consulta y fichas deben usar el mismo codificador

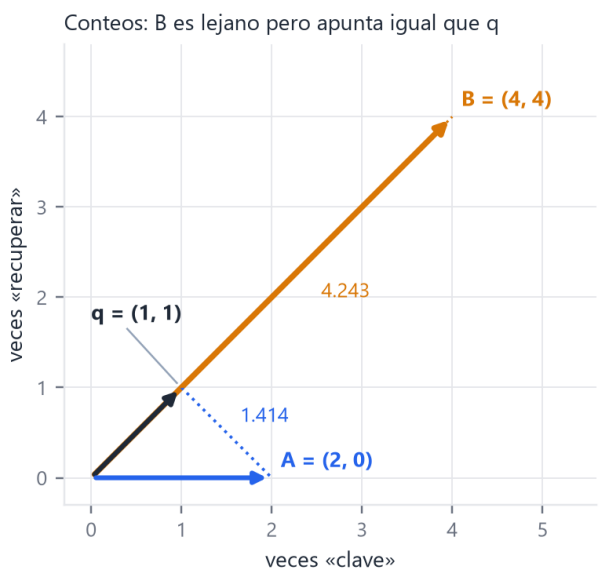
4 Norma, producto interno y coseno, a mano

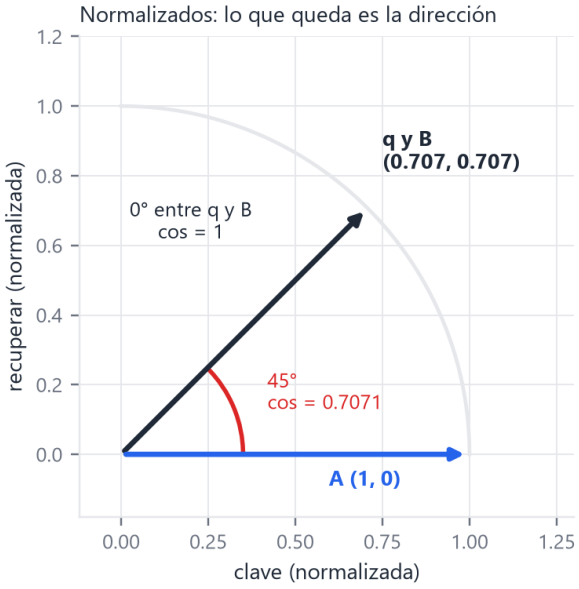
Tres operaciones con vectores. **Producto interno**: multiplicar coordenada a coordenada y sumar. **Norma**: la raíz de la suma de los cuadrados (la longitud). **Coseno**: el producto interno dividido por el producto de las normas; vale 1 si apuntan igual, 0 si son perpendiculares y −1 si son opuestos. Por eso el coseno **no cambia si multiplicas un vector por un número positivo**: solo mira la dirección.

Ejemplo resuelto · q = (3, 4), con ||q|| = √(9 + 16) = 5 (ejemplo propio)

Vector	q·v	‖v‖	Coseno	Distancia a q
a = (6, 8)	50	10	1.00	5.000
b = (4, 3)	24	5	0.96	1.414
c = (−4, 3)	0	5	0.00	7.071
d = (−3, −4)	−25	5	−1.00	10.000

Lectura: a = 2q tiene coseno 1 aunque está a distancia 5 (lejos); b = (4, 3) está a solo 1.414 y su coseno es 24/25 = 0.96; c es perpendicular (3·(−4) + 4·3 = 0); d apunta al lado contrario (coseno −1). La distancia y el coseno responden preguntas distintas. **Error común**: creer que mayor producto interno significa más parecido: a tiene producto 50 y b solo 24, pero buena parte de ese 50 viene del tamaño de a (es 2q); el producto mezcla dirección con tamaño y el coseno las separa.





Lo que debes recordar

- Producto interno = Σ q_j z_j; norma = √(Σ z_j^2)
- Coseno = producto interno / (norma · norma); no depende del largo
- Un coseno de 0.8 no es 80 % de probabilidad

5 De los puntajes a un ranking y al rango de la ficha correcta

Para cada consulta hay **un puntaje por ficha** (una fila de la matriz S = Q D^T). Se ordenan de mayor a menor; si dos puntajes empatan, el programa del profesor desempata por el identificador de la ficha. El **rango r** es la posición que ocupa la ficha correcta. Una fila de ceros (consulta sin ninguna palabra en el vocabulario) no entrega ranking: el programa le asigna aporte 0. Siempre hay una ficha en el primer lugar, aunque ninguna responda la pregunta.

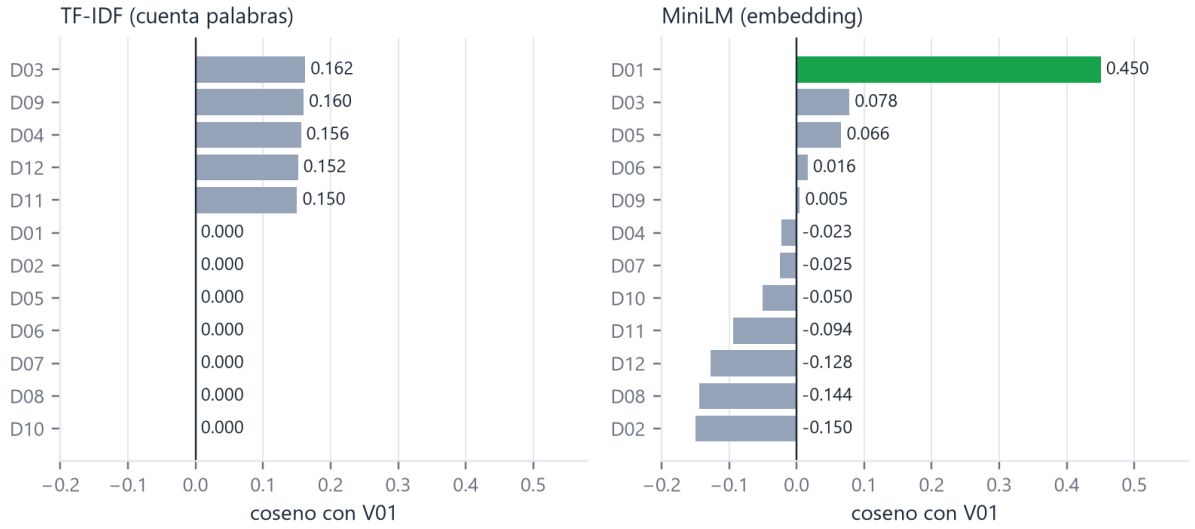
Ejemplo resuelto · una consulta contra 6 fichas, relevante F4 (ejemplo propio)

Posición	Ficha	Puntaje	Comentario
1	F2	0.41	empate con F4: F2 va antes por identificador
2	F4	0.41	relevante
3	F6	0.33	
4	F1	0.12	
5	F3	0.07	
6	F5	−0.05	

Lectura: F2 y F4 empatan en 0.41; por identificador F2 queda primera y F4 segunda: r = 2, rango recíproco 1/r = 0.50. Con k = 1 la consulta no sería un acierto; con k = 2 sí. **Error común**: contar mal la posición cuando hay negativos: −0.05 es menor que 0.07; los puntajes negativos van al final.

En la clase: V02 «No pude asistir, ¿puedo ver el video después?» recibe de MiniLM este orden: D06 (0.236), D02 (0.147), D01 (0.131); la ficha correcta, D02, ocupa el lugar 2 (diapositiva 35).

V01 «Olvidé mi clave…»: la ficha correcta (D01, verde) queda 6.ª con TF-IDF y 1.ª con MiniLM



Lo que debes recordar

- Ranking = ordenar la fila de puntajes de mayor a menor
- r = posición de la ficha relevante; empates por identificador
- El primer lugar existe siempre: no prueba que responda

6 Hit@k y MRR: cómo medir el buscador y elegir k

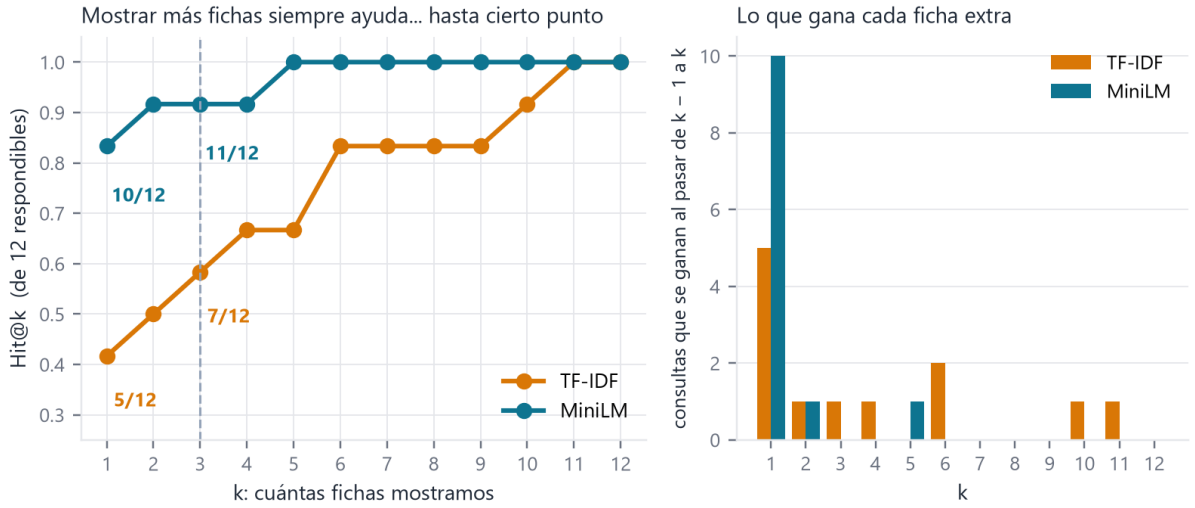
Con las consultas de validación, cuya ficha correcta se conoce, se calcula el rango r de cada una. **Hit@k** es la proporción de consultas con r ≤ k (¿estaba entre las k primeras?). **MRR** es el promedio de 1/r: da 1 al primer lugar, 0.5 al segundo, 0.333 al tercero… y por eso distingue un primer lugar de un segundo, cosa que Hit@k no hace. Como el MRR usa el ranking completo, **cambiar k no lo modifica**. Se calculan solo con las consultas que tienen respuesta.

Ejemplo resuelto · 8 consultas con rangos 1, 3, 2, 1, 6, 1, 4, 2 (ejemplo propio)

Métrica	Cálculo	Resultado
Hit@1	rangos ≤ 1: 1, 1, 1 → 3 de 8	3/8 = 0.375
Hit@3	rangos ≤ 3: 1, 3, 2, 1, 1, 2 → 6 de 8	6/8 = 0.750
Hit@5	rangos ≤ 5: todos menos el 6 → 7 de 8	7/8 = 0.875
MRR	(1 + 1/3 + 1/2 + 1 + 1/6 + 1 + 1/4 + 1/2) ÷ 8	4.7500 ÷ 8 = 0.5938

Lectura: pasar de k = 1 a k = 3 duplica los aciertos, pero 6 y 4 siguen fuera hasta que k llegue a 4 y 6. Elegir k es un equilibrio: más candidatos significan más lectura para quien usa el buscador. **Error común**: promediar los rangos (1 + 3 + 2 + … ÷ 8) en lugar de promediar 1/r; el MRR promedia los recíprocos.

En la clase (12 consultas respondibles): Hit@1 = 5/12 para TF-IDF y 10/12 para MiniLM; Hit@3 = 7/12 y 11/12; MRR = 0.551 y 0.892. MiniLM alcanza 12/12 con k = 5 y TF-IDF con k = 11 (nuestro cálculo).



Lo que debes recordar

- Hit@k = proporción con r ≤ k; MRR = promedio de 1/r
- MRR no depende de k; Hit@k sí
- Se evalúan solo las consultas con respuesta, y k se elige con validación

7 Consultas sin respuesta, cortes de aceptación y límites

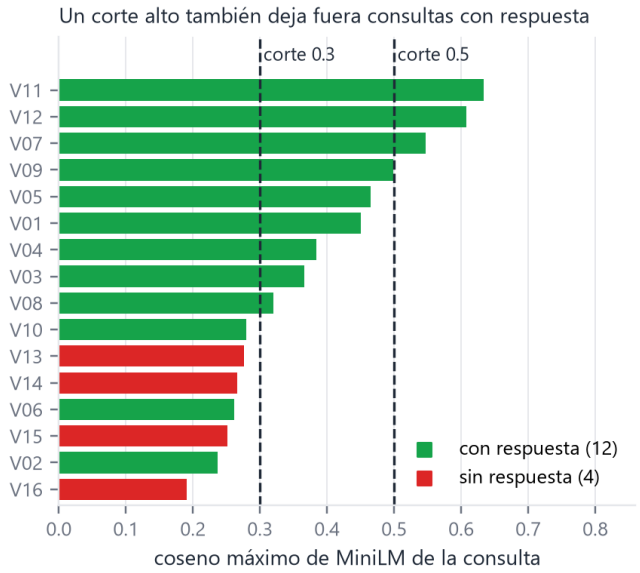
Una consulta puede no tener ninguna ficha que la responda (por ejemplo «¿Hay estacionamiento gratuito?»). El ranking igual entrega un primer lugar. Una solución es aceptar una consulta solo si su **coseno máximo** alcanza un **corte** c. Tiene un costo: un corte alto rechaza también consultas que sí tenían respuesta; uno bajo acepta consultas sin respuesta. Es la misma lógica de un umbral de clasificación (Clases 38 y 41). Y aun aceptada y bien recuperada, la ficha puede ser **insuficiente**: recuperar no es responder.

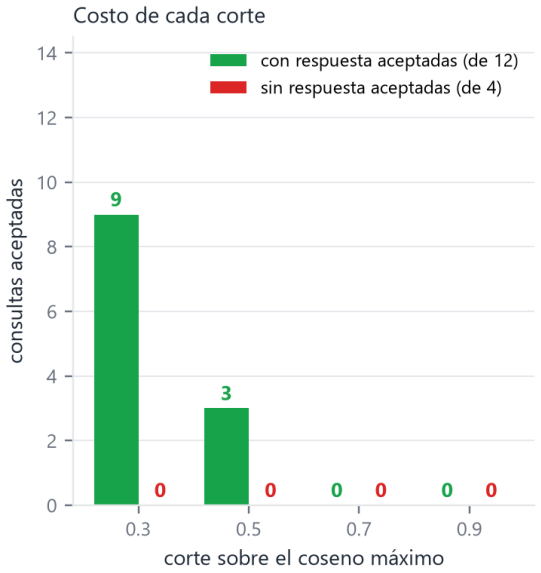
Ejemplo resuelto · 7 consultas con su coseno máximo (ejemplo propio)

Consulta	Coseno máximo	Situación	¿Acepta c = 0.40?	¿Acepta c = 0.30?
Q1	0.62	r = 1	sí	sí
Q2	0.48	r = 2	sí	sí
Q3	0.35	r = 4	no	sí
Q4	0.22	r = 1	no	no
Q5	0.41	sin respuesta	sí	sí
Q6	0.30	sin respuesta	no	sí
Q7	0.18	sin respuesta	no	no

Lectura: con c = 0.40 se aceptan Q1, Q2 (con respuesta) y Q5 (sin respuesta): 2 de 4 respondibles y 1 de 3 sin respuesta. Bajando a c = 0.30 se acepta además Q3 y Q6 (≥ 0.30): 3 de 4 respondibles, pero 2 de 3 sin respuesta. Ningún corte separa perfecto aquí porque Q4 (con respuesta) tiene 0.22 y Q5 (sin respuesta) 0.41. **Error común**: leer el coseno como probabilidad (0.62 no es «62 % seguro»).

En la clase: con c = 0.3 se aceptan 9 de 12 consultas respondibles y 0 de 4 sin respuesta; con c = 0.5, 3 y 0 (diapositiva 42). Las consultas sin respuesta llegan a un coseno máximo de 0.276; V02 solo a 0.236. Los cortes son ilustrativos y 4 consultas sin respuesta son pocas para generalizar. **Códigos exactos**: D11 (A17) y D12 (A71) son casi iguales; con MiniLM su coseno es 0.891. En V11 y V12 la ficha correcta quedó primera, pero el margen de V12 fue de solo 0.060: conviene revisar el código exacto antes de entregar una fecha.



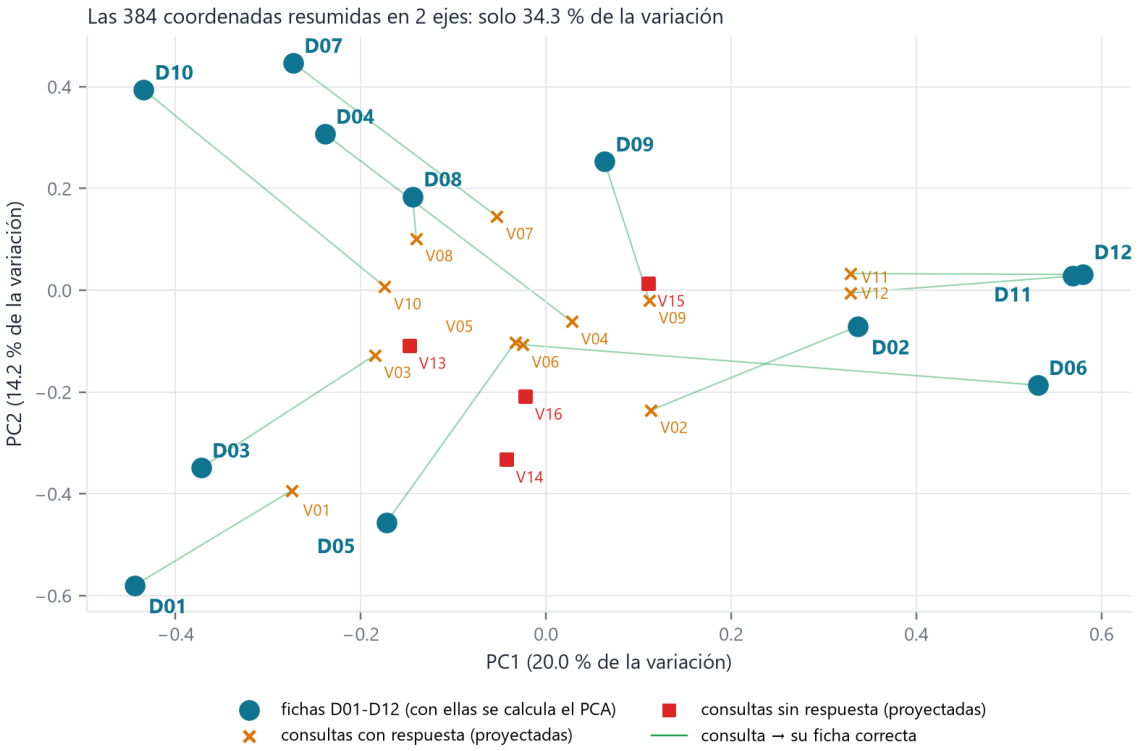


Lo que debes recordar

- Aceptar (corte), recuperar (r ≤ k) y responder (contenido suficiente) son tres criterios distintos
- Un corte alto pierde consultas respondibles; uno bajo acepta consultas sin respuesta
- Los códigos exactos (A17/A71) exigen revisar el dato, no solo el coseno

8 El plano PCA y para qué más sirven los embeddings

No se pueden dibujar 384 coordenadas. El **PCA** las resume en dos ejes: en las fichas, PC1 explica 20.0 % y PC2 14.2 % de la variación, juntos 34.3 %. El dibujo sirve para explorar (D11 y D12 aparecen muy cerca), pero omite casi dos tercios de la variación y **no se usa para ordenar**: el ranking usa las 384 coordenadas. El mismo embedding puede ser la entrada de un clasificador (por ejemplo, spam o no spam con sus etiquetas), de un agrupamiento o de una búsqueda de vecinos: cada tarea se evalúa con su propio criterio.



Lo que debes recordar

- PCA solo dibuja: no calcula el ranking
- Cambiar de codificador exige recalcular fichas y consultas en el mismo espacio
- Cambiar la tarea cambia qué hay que evaluar